



Jogo de Truco

Tutorial Completo do Código

Explicação técnica da arquitetura em HTML, CSS e JavaScript, mostrando como o jogo foi organizado, por que cada módulo existe e onde localizar as principais rotinas no código.

40 CARTAS	ATÉ 12 PONTOS	3 NÍVEIS DE IA
Vira e manilha variável	Truco, Seis, Nove e Doze	Fácil, Normal e Difícil

JOGO DE TRUCO

Sumário

01. Arquitetura geral
02. Estrutura de arquivos
03. HTML e telas
04. CSS e layout responsivo
05. Módulo de regras
06. Baralho e embaralhamento
07. Vira e manilha
08. Cálculo da força das cartas
09. Apostas e nomes dos pedidos
10. Vencedor da vaza
11. Vencedor da mão e empates
12. Módulo de Inteligência Artificial
13. Avaliação numérica da mão
14. Escolha de carta da IA
15. Carta fechada da IA
16. Pedidos e blefes da IA
17. Respostas da IA aos pedidos
18. Módulo principal da partida
19. Estado da classe PartidaTrucoWeb
20. Criação de uma nova mão
21. Mão de 11
22. Validação das ações do jogador
23. Jogada e rotação de turnos
24. Resolução de vaza
25. Encerramento da mão e partida
26. Fluxo automático das IAs
27. Sistema de Truco, Seis, Nove e Doze
28. Janelas de decisão
29. Modo 4 Jogadores
30. Renderização da mesa e cartas
31. Placar, status e histórico
32. Ranking com localStorage
33. Sons
34. Eventos e atalhos
35. Responsividade
36. Mapa de funções e linhas
37. Fluxo completo de execução
38. Decisões de projeto e manutenção

1. Arquitetura geral

O Jogo de Truco foi organizado em uma arquitetura web simples e separada por responsabilidades. O HTML define a estrutura visual; o CSS controla aparência e adaptação a diferentes tamanhos de tela; e o JavaScript foi dividido entre regras, Inteligência Artificial e coordenação geral da partida.

A separação mais importante é esta: regras.js sabe como o Truco funciona, ia.js sabe como os oponentes tomam decisões e app.js sabe como a partida acontece e como a interface reage.

2. Estrutura de arquivos

Arquivo/Pasta	Responsabilidade
index.html	Telas do menu e da partida, mesa, placar, botões, ranking e modal.
css/estilo.css	Tema visual, cartas, mesa, placar, modais e responsividade.
js/regras.js	Baralho, manilha, força das cartas, apostas e desempates.
js/ia.js	Avaliação de mão, escolha de cartas, blefe, carta fechada e respostas.
js/app.js	Estado da partida, turnos, UI, sons, Mão de 11, ranking e eventos.
sons/*.wav	Efeitos sonoros usados em ações e resultados.
ABRIR_JOGO.bat	Atalho para abrir o jogo no ambiente Windows.

3. HTML e telas

O arquivo index.html possui duas telas principais. A tela de menu começa na linha 76 e contém nome do jogador, modo, dificuldade e ranking. A tela da partida começa na linha 457 e contém placar, mesa, participantes, Vira, área das jogadas, mão do jogador, valor em disputa e botões de ação.

O sistema de modal reutilizável fica a partir da linha 1115. Em vez de criar uma janela diferente para cada decisão, o código mantém uma estrutura única e altera título, texto, ícone e botões conforme a situação.

4. CSS e layout responsivo

O arquivo css/estilo.css concentra toda a identidade visual. A mesa, o feltro, as cartas, os assentos, o placar, os botões e as janelas de decisão são construídos com CSS, sem necessidade de imagens externas para representar as cartas.

Faixa aproximada	Conteúdo
289-984	Menu principal e ranking.
1180-2408	Placar, mesa, assentos, cartas, ações e avisos.
2559-2633	Modal e primeiras regras responsivas.
5361-6500	Janelas especiais de decisão e cartas da dupla.
7106-7396	Indicador PARTIDA VALENDO e ajustes de tela.

Os blocos @media reorganizam tamanhos, posições e espaçamentos conforme largura e altura disponíveis. Isso evita que a mesa dependa de um único tamanho de monitor.

5. Módulo de regras

js/regras.js é o núcleo determinístico. Ele não decide estratégia e não manipula a tela. Recebe cartas e estados simples e devolve resultados. Isso reduz o risco de misturar regra de jogo com interface.

As constantes principais estão nas linhas 29-81: ordem dos valores, naipes, nomes dos naipes, sequência de apostas e pontuação necessária para vencer.

6. Baralho e embaralhamento

A função `criarBaralho()`, linhas 89-142, percorre os dez valores e os quatro naipes para formar 40 objetos de carta. Cada objeto possui valor e naipe.

`embaralhar()`, linhas 151-220, cria uma cópia do array e aplica uma troca aleatória de posições de trás para frente. O objetivo é entregar um baralho novo sem alterar o array recebido.

7. Vira e manilha

`valorManilha(vira)`, linhas 256-289, procura o valor da Vira em `ORDEM_VALORES` e devolve o valor seguinte. O operador de resto faz a sequência voltar ao início quando a Vira é 3, transformando o 4 em manilha.

Essa função foi isolada porque a mesma informação é usada pela comparação das cartas, pela interface e pela IA.

8. Cálculo da força das cartas

`forcaCarta()`, linhas 300-344, converte uma carta em um número comparável. Cartas normais usam a posição na sequência. Manilhas recebem valores a partir de 100, garantindo que sempre sejam maiores que cartas comuns.

A expressão `100 + NAIPES.indexOf(carta.naipe)` faz as manilhas seguirem a ordem Ouros, Espadas, Copas e Paus em valores crescentes. Portanto, Paus recebe a maior força.

`compararCartas()`, linhas 356-397, utiliza essas forças e retorna 1, 0 ou -1 para indicar maior, empate ou menor.

9. Apostas e nomes dos pedidos

`proximoValorAposta()`, linhas 406-443, busca o valor atual em [1, 3, 6, 9, 12] e devolve o próximo. Em 12, devolve null porque não existe novo aumento.

`nomePedido()`, linhas 452-481, transforma valores em rótulos legíveis: NORMAL, TRUCO, SEIS, NOVE e DOZE. Isso evita repetir regras de nomenclatura em vários pontos da interface.

10. Vencedor da vaza

`vencedorVaza()`, linhas 644-838, recebe todas as jogadas da vaza. Primeiro remove as cartas marcadas como fechadas. Depois calcula a maior força entre as cartas abertas.

Se nenhuma carta estiver aberta, a vaza empata. Se as maiores cartas pertencerem a equipes diferentes, também há empate. Caso as maiores cartas estejam na mesma equipe, essa equipe vence a vaza.

11. Vencedor da mão e empates

`vencedorMao()`, linhas 492-633, recebe a sequência dos vencedores das vazas. O valor null representa empate. A função implementa as combinações que permitem encerrar a mão antes da terceira vaza ou decidir o resultado final após três vazas.

A função também recebe `equipeMao`. Ela é usada quando todas as condições terminam empatadas e a vantagem precisa retornar para a equipe que abriu aquela mão.

12. Módulo de Inteligência Artificial

`js/ia.js` não altera a partida diretamente. Ele recebe dados da mesa e devolve decisões. Essa separação permite testar ou modificar o comportamento das IAs sem reescrever a lógica de turnos.

A função auxiliar `C()` referencia `window.TrucoCore`, permitindo que a IA reutilize as mesmas regras de força e manilha usadas pelo jogo.

13. Avaliação numérica da mão

`avaliarMao()`, linhas 93-244, converte as cartas disponíveis em uma nota. Manilhas recebem peso muito alto; 3, 2, Ás e outras cartas fortes recebem pesos progressivamente menores.

A função `ruido()`, linhas 31-78, acrescenta uma pequena variação aleatória. No nível Fácil a amplitude é maior; no Difícil é pequena. Isso evita que todas as decisões sejam idênticas e cria diferenças reais entre os níveis.

14. Escolha de carta da IA

`escolherCarta()`, linhas 340-691, ordena as cartas pela força e observa a melhor jogada já colocada na mesa. Se a própria equipe estiver vencendo, tende a economizar. Se estiver perdendo e puder superar a carta atual, procura a menor carta que consiga vencer.

No nível Difícil, existem decisões adicionais para a primeira vaza e para placares apertados. No Fácil, há maior chance de escolher aleatoriamente, simulando erros de leitura.

15. Carta fechada da IA

`deveJogarFechada()`, linhas 709-872, só considera carta fechada da segunda vaza em diante. A chance varia por dificuldade e aumenta em situações em que a equipe já está vencendo ou quando a IA não possui carta capaz de superar a mesa.

Quando a IA decide descartar fechada, `escolherDescarteFechado()`, linhas 883-937, procura a carta mais fraca para preservar recursos fortes.

16. Pedidos e blefes da IA

`devePedirAumento()`, linhas 956-1074, combina a nota da mão com fatores como vitória na primeira vaza e situação do placar. Depois compara a nota com um limiar que muda de acordo com a dificuldade.

A função também possui uma probabilidade explícita de blefe. No Difícil, o valor de blefe é maior do que no Fácil. Isso faz com que pedidos não revelem automaticamente uma mão forte.

17. Respostas da IA aos pedidos

`responderPedido()`, linhas 1092-1223, calcula uma nota e decide entre correr, aceitar ou aumentar. A decisão considera força das cartas, resultado da primeira vaza, pontuação e dificuldade.

Se a nota for baixa, a tendência é correr. Acima de um limiar intermediário, aceita. Com nota alta e existência de próximo valor, pode responder com novo aumento.

18. Módulo principal da partida

js/app.js conecta regras, IA e interface. Ele contém utilitários para temporizadores, sons, troca de tela, modais, avisos, ranking, renderização e a classe que representa a partida.

Esse arquivo também registra os eventos de clique e teclado e inicializa o jogo quando o HTML termina de carregar.

19. Estado da classe PartidaTrucoWeb

A classe PartidaTrucoWeb começa na linha 1676. O construtor, linhas 1688-1990, cria os participantes conforme o modo e inicializa todo o estado: placar, jogador da mão, número da mão, valor em disputa, Vira, vazas, jogadas atuais, turno, bloqueios, carta fechada e pedido pendente.

No Mano a Mano são criados dois participantes. No modo de quatro jogadores são criados quatro participantes com equipes alternadas: jogador humano e IA Parceira na equipe 0; IA 2 e IA 3 na equipe 1.

20. Criação de uma nova mão

novaMao(), linhas 2066-2280, cria e embaralha o baralho, limpa as mãos, distribui três cartas para cada participante, escolhe a Vira e prepara os estados de turno e vazas.

O valor inicial é definido como 1 em uma mão normal ou 3 se maoDeOnze() detectar uma equipe com 11 pontos. Depois a função toca o som de nova mão, renderiza a mesa e direciona o fluxo para Mão de 11 ou para o turno normal.

21. Mão de 11

tratarMaoDeOnze(), linhas 2287-2511, identifica qual equipe está com 11. Se for a equipe do humano, abre uma janela que mostra as cartas necessárias para decidir entre correr por 1 ponto ou jogar valendo 3.

iaCorreMaoOnze(), linhas 2519-2607, calcula a força média das cartas da equipe de IA e compara com um limite dependente da dificuldade. Assim a decisão não é puramente aleatória.

22. Validação das ações do jogador

podeJogarHumano(), linha 2626, exige mão ativa, ausência de bloqueio, ausência de pedido pendente e turno do jogador. podeFechada(), linha 2643, acrescenta a condição de estar pelo menos na segunda vaza.

podePedir(), linhas 2651-2676, impede aumentos na Mão de 11, bloqueia pedidos quando já existe uma decisão pendente, impede a mesma equipe de pedir consecutivamente e verifica se existe próximo valor de aposta.

23. Jogada e rotação de turnos

jogarHumano(), linhas 2684-2732, valida a vez do jogador, lê o estado de carta fechada e encaminha a ação para jogarCarta().

jogarCarta(), linhas 2745-2937, remove a carta da mão do participante, registra a jogada com jogador, equipe, carta e flag de fechada, toca o efeito sonoro e avança o turno. Quando todos jogaram, chama a resolução da vaza.

24. Resolução de vaza

`resolverVaza()`, linhas 2944-3236, bloqueia temporariamente a mesa, chama `TrucoCore.vencedorVaza()`, registra o histórico e exibe o resultado com som e aviso.

Depois consulta `vencedorMao()`. Se a mão já estiver decidida, chama `encerrarMao()`. Caso contrário, limpa as jogadas da vaza e define quem abre a próxima: o vencedor da vaza ou, em caso de empate, quem havia aberto a vaza anterior.

25. Encerramento da mão e partida

`encerrarMao()`, linhas 3247-3529, soma os pontos à equipe vencedora e verifica se ela alcançou 12. Se a partida terminou, toca o som final, registra o ranking e abre a janela de resultado.

Se ainda não houver vencedor da partida, o índice da mão avança circularmente e uma nova mão é criada após o intervalo visual.

26. Fluxo automático das IAs

`processarTurno()`, linhas 3536-3888, é o controlador dos turnos automatizados. Ele só age quando a mão está ativa e não existe bloqueio nem pedido pendente. Se o turno pertencer ao humano, encerra sem fazer nada.

Para uma IA, a função aguarda um pequeno tempo, verifica se deve pedir aumento, decide se joga fechada, escolhe o índice da carta e finalmente chama `jogarCarta()`.

27. Sistema de Truco, Seis, Nove e Doze

`pedirAumento()`, linhas 3913-4054, cria `pedidoPendente` contendo novo valor, equipe solicitante, jogador solicitante e valor anterior. O valor anterior é essencial porque é ele que será entregue ao solicitante se o adversário correr.

Se o pedido vier da equipe adversária, o humano recebe a janela de resposta. Se vier da equipe humana, a resposta é calculada automaticamente pela IA adversária.

`respostaAutomatica()`, linhas 4082-4605, trata corrida, aceitação ou contra-aumento das IAs.

`responderHumano()`, linhas 4807-4961, aplica as mesmas possibilidades quando a decisão pertence ao jogador.

28. Janelas de decisão

`abrirModal()`, linhas 664-897, é um componente genérico. Ele recebe rótulo, título, HTML, ícone, botões e tipo. A mesma infraestrutura serve para ajuda, confirmação, Mão de 11, pedidos de aposta e resultado final.

`htmlCartasDaDecisao()`, linhas 1457-1670, monta o painel de cartas mostrado nas decisões. No modo de duplas, inclui as cartas da IA Parceira, além de Vira e manilha.

`modalResponderPedido()`, linhas 4612-4799, usa esse painel e cria os botões CORRER, ACEITAR e, quando possível, AUMENTAR.

29. Modo 4 Jogadores

A criação dos quatro participantes ocorre no construtor. A ordem é jogador humano, IA 2, IA Parceira e IA 3. Como as equipes são alternadas, a rotação simples `(turno + 1) % jogadores.length` produz a sequência horária correta e mantém os parceiros em posições opostas.

A renderização da mesa posiciona a IA Parceira no topo e as duas IAs adversárias nas laterais. No Mano a Mano, apenas o oponente do topo permanece visível.

30. Renderização da mesa e cartas

`criarCartaHTML()`, linhas 5040-5112, gera o HTML de uma carta aberta ou fechada. `renderizarAssento()`, linhas 5113-5250, exibe nome, equipe, quantidade de cartas e versos das cartas das IAs.

`renderizarMesa()`, linhas 5258-5519, organiza assentos, Vira, manilha, valor da mão e cartas que já estão no centro. `renderizarMao()`, linhas 5527-5642, cria os botões das cartas humanas e registra clique e clique direito.

31. Placar, status e histórico

`renderizarPlacar()`, linhas 5650-5743, preenche pontos, nomes das equipes e marcadores. `renderizarAcoesStatus()`, linhas 5751-6044, atualiza mensagens, botão de Truco, carta fechada e indicador PARTIDA VALENDO.

`renderizarVazas()`, linhas 6052-6240, constrói os três blocos do histórico. `renderizarTudo()`, linha 6258, centraliza as chamadas de renderização para manter a interface sincronizada com o estado.

32. Ranking com localStorage

O ranking usa a chave `trucoPaulistaRanking`. `obterRanking()`, linhas 1154-1181, lê e converte o JSON. `salvarRanking()`, linha 1200, grava novamente o array.

`registrarRanking()`, linhas 1211-1334, procura o jogador pelo nome normalizado. Vitória soma 10 pontos; derrota reduz 10 sem permitir valor abaixo de zero. Também mantém partidas, vitórias e derrotas.

`atualizarRankingMenu()`, linhas 1342-1429, ordena por pontos, depois vitórias e partidas, e exibe apenas os cinco primeiros.

33. Sons

O projeto inclui arquivos WAV separados para eventos como aceitar, aumentar, carta, carta fechada, chamada, correr, empate, nova mão, vitória/derrota de vaza, mão e partida. A função `tocar()`, iniciada na linha 312 de `app.js`, centraliza a reprodução e respeita o estado do botão de som.

Centralizar o áudio em uma função evita que cada regra precise conhecer detalhes de reprodução ou caminhos de arquivo.

34. Eventos e atalhos

`configurarEventos()`, linhas 6414-6749, liga os botões do menu, ajuda, som, Truco, carta fechada, ranking e confirmações às funções correspondentes. O mesmo bloco registra o evento `keydown` para T, C, S, H, R e ESC.

A inicialização final ocorre em `DOMContentLoaded`, linhas 6761-6785. Ela configura os eventos, seleciona Mano a Mano como modo inicial e carrega o ranking do navegador.

35. Responsividade

A responsividade não depende de JavaScript. O CSS contém várias faixas de mídia para controlar largura, altura, disposição da mesa, tamanho das cartas, painel de ações e modais. Em telas menores, a estrutura passa a utilizar organização mais vertical.

Essa escolha mantém a lógica do jogo independente do dispositivo: o JavaScript trabalha com o mesmo estado, enquanto o CSS decide apenas como apresentá-lo.

36. Mapa de funções e linhas

Arquivo	Função	Linhas	Responsabilidade
regras.js	criarBaralho()	89-142	Monta as 40 cartas.
regras.js	embaralhar()	151-220	Randomiza o baralho.
regras.js	valorManilha()	256-289	Calcula o valor seguinte à Vira.
regras.js	forcaCarta()	300-344	Converte carta em força numérica.
regras.js	vencedorMao()	492-633	Resolve vazas e empates.
regras.js	vencedorVaza()	644-838	Compara cartas abertas da vaza.
ia.js	avaliarMao()	93-244	Calcula a nota estratégica.
ia.js	escolherCarta()	340-691	Seleciona a carta da IA.
ia.js	deveJogarFechada()	709-872	Decide descarte oculto.
ia.js	devePedirAumento()	956-1074	Decide Truco ou aumento.
ia.js	responderPedido()	1092-1223	Corre, aceita ou aumenta.
app.js	novaMao()	2066-2280	Distribui cartas e reinicia estados.
app.js	tratarMaoDeOnze()	2287-2511	Controla a Mão de 11.
app.js	jogarCarta()	2745-2937	Registra carta e avança turno.
app.js	resolverVaza()	2944-3236	Calcula e registra a vaza.
app.js	encerrarMao()	3247-3529	Pontua e verifica fim da partida.
app.js	processarTurno()	3536-3888	Executa turnos das IAs.
app.js	pedirAumento()	3913-4054	Cria um pedido de aposta.
app.js	respostaAutomatica()	4082-4605	Aplica decisão da IA.
app.js	modalResponderPedido()	4612-4799	Monta a decisão do humano.
app.js	responderHumano()	4807-4961	Aplica correr, aceitar ou aumentar.
app.js	renderizarMesa()	5258-5519	Desenha estado da mesa no DOM.
app.js	renderizarAcoesStatus()	5751-6044	Atualiza ações e valor em disputa.
app.js	configurarEventos()	6414-6749	Conecta interface e teclado.

37. Fluxo completo de execução

O fluxo começa com `DOMContentLoaded`, que prepara eventos e menu. Ao iniciar a partida, `iniciarPartidaComConfiguracaoAtual()` cria `PartidaTrucoWeb` e chama `iniciar()`. Isso leva a `novaMao()`, que distribui as cartas e chama `processarTurno()`.

Cada carta passa por `jogarCarta()`. Ao completar a quantidade de participantes, `resolverVaza()` consulta o módulo de regras. Quando uma mão termina, `encerrarMao()` pontua e inicia a próxima ou encerra a partida.

Pedidos de aposta interrompem temporariamente esse fluxo por meio de `pedidoPendente`. Após aceitar, aumentar ou correr, o estado é liberado e a sequência normal continua.

38. Decisões de projeto e manutenção

A divisão em três módulos JavaScript reduz acoplamento. Uma alteração na força das cartas pode ser feita em `regras.js`; uma mudança de comportamento da IA pode ficar em `ia.js`; e uma alteração de interface ou fluxo pode ficar em `app.js`.

O estado centralizado na classe evita depender do conteúdo visual da página como fonte de verdade. A interface apenas representa o estado atual. Isso é importante porque placar, turno, vazas, pedidos e cartas precisam permanecer sincronizados.

O uso de `localStorage` foi escolhido para o ranking porque não exige servidor. A vantagem é a simplicidade e o funcionamento offline; a consequência é que o ranking pertence somente ao navegador em que as partidas foram realizadas.

Os temporizadores e bloqueios ajudam a impedir cliques enquanto uma vaza está sendo resolvida ou uma IA está pensando. Isso evita que ações concorrentes alterem o estado no meio de uma transição.

Para estudar ou modificar o projeto, a sequência mais segura é: primeiro entender `regras.js`, depois `ia.js` e por último acompanhar o fluxo de `app.js`. Essa ordem parte das regras puras e termina na coordenação completa da interface.